

4강. React Hook(상태 관리)



React

리액트 Hook

■ 리액트 Hook이란?

리액트에서는 상태를 관리할 때 훅을 사용한다. 훅이란 함수형 컴포넌트에서 상태(state)와 생명주기(lifecycle)를 쉽게 관리할 수 있도록 도와주는 리액트에서 제공하는 특별한 함수이다.

주요 함수로는 **useState()**, **useEffect()**, **useRef()** 등이 있다.

```
const [state, setState] = useState(0)
```

- state: 상태 값을 저장하는 변수로 보통 상태 변수라고 부른다.
- setState: 상태를 변경하는 함수로, 상태 변경 함수라고 부른다.

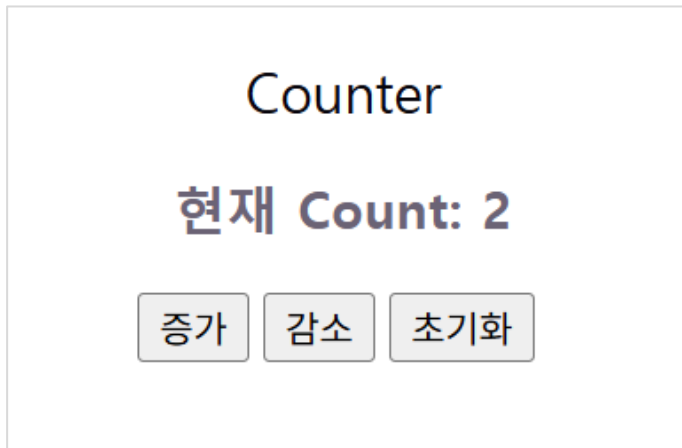
예) count 상태를 변경하는 코드

```
const [count, setCount] = useState(0)
```



리액트 Hook – useState()

- Counter 만들기



리액트 Hook – useState()

- Counter 만들기

```
import { useState } from "react";

const Counter = () => {
  //count 상태 관리
  const [count, setCount] = useState(0);

  //감소 핸들러
  const decrement = () => {
    setCount(count-1);
  }

  //초기화 핸들러
  const reset = () => {
    setCount(0);
  }
}
```



리액트 Hook – useState()

- Counter 만들기

```
return(  
  <div>  
    <h2>Counter</h2>  
    <h3>현재 Count: {count}</h3>  
    { /* 인라인 핸들러 함수 */ }  
    <button onClick={() => setCount(count+1)}>증가</button>  
    <button onClick={decrement}>감소</button>  
    <button onClick={reset}>초기화</button>  
  </div>  
)  
}  
  
export default Counter;
```



리액트 Hook – useState()

- 자동차 연식 업데이트

자동차 컴포넌트

브랜드: 현대

모델: 쏘나타

연식: 2025

연식 업데이트



리액트 Hook – useState()

▪ 자동차 연식 업데이트

```
const Car = () => {  
  const [car, setCar] = useState({  
    brand: '현대',  
    model: '쏘나타',  
    year: 2020  
  });  
  
  const updateYear = () => {  
    /* car 객체의 year 속성만 업데이트하기 위해  
    | 기존의 car 객체를 복사하고 year 속성만 변경 */  
    setCar({ ...car, year: 2025 });  
  };  
  
  return (  
    <div>  
      <h2>자동차 컴포넌트</h2>  
      <p>브랜드: {car.brand}</p>  
      <p>모델: {car.model}</p>  
      <p>연식: {car.year}</p>  
      <button onClick={updateYear}>연식 업데이트</button>  
    </div>  
  );  
}
```



리액트 Hook – useState()

- 음료 추가 – 배열 상태 관리1

음료 추가

현재 음료: 커피, 콜라, 딸기쥬스

음료 추가



리액트 Hook – useState()

▪ 음료 추가 – 배열 상태 관리1

```
import { useState } from "react";

const Drinks = () => {
  //음료 배열 상태 관리 - 초기화
  const [drinks, setDrinks] = useState(['커피', '콜라']);

  const addDrink = () => {
    //기존 drinks 배열에 '딸기쥬스'를 추가하여 새로운 배열 생성
    setDrinks([...drinks, '딸기쥬스'])
  }

  return(
    <div>
      <h2>음료 추가</h2>
      <h4>현재 음료: {drinks.join(', ')}</h4>
      <button onClick={addDrink}>음료 추가</button>
    </div>
  )
}
```



리액트 Hook – useState()

- 음료 추가 – 배열 상태 관리2

음료 추가

아이스 아메리카노
딸기 쥬스
카페라떼



리액트 Hook – useState()

▪ 음료 추가 – 배열 상태 관리2

```
const Drinks2 = () => {  
  const [drinks, setDrinks] = useState([]);  
  //입력 필드의 상태 관리  
  const [inputValue, setInputValue] = useState('');  
  
  const addDrink = () => {  
    const newDrink = inputValue.trim();  
    if(newDrink == ''){  
      alert('음료 이름을 입력하세요');  
      return;  
    }  
    //새 음료를 기존 배열에 추가  
    setDrinks([...drinks, newDrink]);  
    setInputValue(''); //입력 필드 초기화  
  }  
}
```



리액트 Hook – useState()

▪ 음료 추가 – 배열 상태 관리2

```
return (  
  <div>  
    <h2>음료 추가</h2>  
    <input  
      type="text"  
      placeholder="음료 이름을 입력하세요"  
      value={inputValue}  
      onChange={(e) => setInputValue(e.target.value)}  
    />  
    <button onClick={addDrink}>음료 추가</button>  
    { /* <p>입력값: {inputValue}</p> */ }  
  
    <ul>  
      {drinks.map((drink, index) => (  
        <li key={index}>{drink}</li>  
      ))}  
    </ul>  
  </div>  
)  
}
```



리액트 Hook – useState()

- 컴포넌트 분리하고 props 사용하기

```
return (  
  <div>  
    <h2>음료 추가</h2>  
    <input  
      type="text"  
      placeholder="음료 이름을 입력하세요"  
      value={inputValue}  
      onChange={(e) => setInputValue(e.target.value)}  
    />  
    <button onClick={addDrink}>음료 추가</button>  
  
    { /* DrinkList 컴포넌트 연결 - drinks 속성으로 전달 */ }  
    <DrinkList drinks={drinks}/>  
  </div>  
)  
}  
  
export default Drinks2;
```

AddDrink2.jsx



리액트 Hook – useState()

- 컴포넌트 분리하고 props 사용하기

DrinkList.jsx

```
const DrinkList = ({ drinks }) => {  
  return (  
    <ul>  
      {drinks.map((drink, index) => (  
        <li key={index}>{drink}</li>  
      ))}  
    </ul>  
  );  
}  
  
export default DrinkList;
```



할 일 관리(Todo-app)

- 할 일 관리

Todo List

☐ 운동 하기

☒ 영화 보기



할 일 관리(Todo-app)

■ 할 일 관리

```
function App() {  
  const [todos, setTodos] = useState([  
    { id: 1, text: '운동 하기', completed: false },  
    { id: 2, text: '영화 보기', completed: false },  
  ])  
  const [inputValue, setInputValue] = useState('')  
  
  console.log(todos.length); //2  
  
  // 입력값 변경 핸들러  
  const handleInputChange = (e) => {  
    console.log(e.target.value);  
    setInputValue(e.target.value)  
  }  
}
```



할 일 관리(Todo-app)

■ 할 일 관리

```
// 할 일 추가
const handleAddTodo = () => {
  if (inputValue.trim() !== '') {
    // 새로운 할 일 객체 생성
    const newTodo = {
      id: todos.length + 1,
      text: inputValue,
      completed: false,
    }
    // 새로운 할 일을 기존 할 일 목록에 추가
    setTodos([...todos, newTodo])
    setInputValue('') // 입력 필드 초기화
  }
}
```



할 일 관리(Todo-app)

■ 할 일 관리

```
// 할 일 완료 체크
const handleToggleComplete = (id) => {
  /* 해당 id를 가진 할 일의 completed 상태를 토글
   | todos 배열을 순회하면서 id가 일치하는 할 일의 completed 값을 반전시킴
   | id가 일치하지 않는 할 일은 그대로 유지*/
  setTodos(
    todos.map((todo) =>
      todo.id === id ? { ...todo, completed: !todo.completed } : todo
    )
  )
}

// 할 일 삭제
const handleDeleteTodo = (id) => {
  /* 해당 id를 가진 할 일을 삭제
   | todos 배열을 필터링하여 id가 일치하지 않는 할 일만 남김*/
  setTodos(todos.filter((todo) => todo.id !== id))
}
```



할 일 관리(Todo-app)

- 할 일 관리

```
return (  
  <>  
    <div className='container'>  
      <h2>Todo List</h2>  
      <input  
        type="text"  
        value={inputValue}  
        onChange={handleInputChange}  
        placeholder="할 일을 입력하세요"  
      />  
      <button onClick={handleAddTodo}>추가</button>  
    </>  
  )
```



할 일 관리(Todo-app)

■ 할 일 관리

```
    { /* 할 일 목록 */  
      <ul className='todo-list'>  
        {todos.map((todo) => (  
          <li key={todo.id} className={todo.completed ? 'completed' : ''}>  
            <input  
              type="checkbox"  
              checked={todo.completed}  
              onChange={() => handleToggleComplete(todo.id)}  
            />  
            {todo.text}  
            <button onClick={() => handleDeleteTodo(todo.id)}>삭제</button>  
          </li>  
        ))}  
      </ul>  
    </div>  
  </>  
)  
}
```



할 일 관리(Todo-app)

■ 할 일 관리

```
body {  
  background-color:  #f1fff0;  
  margin: 0;  
  padding: 20px;  
}  
  
.container{  
  max-width: 400px;  
  margin: 0 auto;  
  background-color: white;  
  padding: 20px;  
  border-radius: 8px;  
  text-align: center;  
}  
  
.container h2{  
  color:  #00f;  
}
```

App.css



할 일 관리(Todo-app)

■ 할 일 관리

```
.todo-list{
  list-style: none;
  padding: 0;
  margin-top: 30px;
}
.todo-list li{
  margin-bottom: 10px;
}
.todo-list li input[type="checkbox"]{
  margin-right: 10px;
}
.completed{
  text-decoration: line-through;
  color: gray;
}
button{
  margin-left: 10px;
}
```

App.css



할 일 관리(Todo-app)

- TodoList 컴포넌트 분리하기

```
return (  
  <>  
    <div className='container'>  
      <h2>Todo List</h2>  
      <input  
        type="text"  
        value={inputValue}  
        onChange={handleInputChange}  
        placeholder="할 일을 입력하세요"  
      />  
      <button onClick={handleAddTodo}>추가</button>  
  
      { /* 할 일 목록 */ }  
  
      <TodoList  
        todos={todos}  
        onToggleComplete={handleToggleComplete}  
        onDeleteTodo={handleDeleteTodo}  
      />  
    </div>  
  </>  
)
```



할 일 관리(Todo-app)

▪ TodoList 컴포넌트 만들기

```
function TodoList({ todos, onToggleComplete, onDeleteTodo }) {  
  return (  
    <ul className='todo-list'>  
      {todos.map((todo) => (  
        <li key={todo.id} className={todo.completed ? 'completed' : ''}>  
          <input  
            type="checkbox"  
            checked={todo.completed}  
            onChange={() => onToggleComplete(todo.id)}  
          />  
          {todo.text}  
          <button onClick={() => onDeleteTodo(todo.id)}>삭제</button>  
        </li>  
      ))}  
    </ul>  
  )  
}
```



Hook – useEffect()

▪ useEffect()

useEffect 혹은 함수형 컴포넌트에서 ****사이드 이펙트(side effect)****를 처리하기 위해 사용하는 핵심 훅입니다.

- 데이터 가져오기 (API 호출)
- 이벤트 등록
- 타이머 설정
- DOM 직접 조작

```
useEffect(() => {  
  // 실행할 코드  
}, [의존성]);
```

```
useEffect(() => {  
  console.log("count 변경됨");  
}, [count]);
```



Hook – useEffect()

- **useEffect()**

The screenshot shows a web application on the left and the React DevTools console on the right. The web application, titled '사용자 정보' (User Information), has two input fields: the first contains '박상희' and the second contains '3'. Below the inputs, it displays '이름: 박상희' and '나이: 3'. The DevTools console on the right has the 'Console' tab selected. It shows a series of log messages: 'Download the React DevTools for a', '렌더링 ..', '이름: , 나이: 1', '렌더링 ..', '이름: , 나이: 1', '렌더링 ..', '이름: , 나이: 2', '렌더링 ..', and '이름: , 나이: 3'. This sequence demonstrates how the `useEffect` hook runs after each render, allowing the UI to be updated based on state changes.



Hook – useEffect()

- useEffect()

```
import { useEffect, useState } from 'react';

const User = () => {
  const [name, setName] = useState("");
  const [age, setAge] = useState(1);

  // [] -> 처음 렌더링될 때만 실행
  // [age] -> age가 변경될 때마다 실행
  useEffect(() => {
    console.log("렌더링..");
    console.log(`이름: ${name}, 나이: ${age}`);
  }, [age]);

  // 이름을 변경하는 이벤트 핸들러 함수
  const onChangeName = (e) => {
    setName(e.target.value);
  }

  // 나이를 변경하는 이벤트 핸들러 함수
  const onChangeAge = (e) => {
    setAge(Number(e.target.value));
  }
}
```



Hook – useEffect()

- **useEffect()**

```
return (  
  <div>  
    <h2>사용자 정보</h2>  
    <input  
      type="text"  
      value={name}  
      onChange={onChangeName}  
    />  
    <br />  
    <input  
      type="number"  
      value={age}  
      onChange={onChangeAge}  
    />  
    <p>이름: {name}</p>  
    <p>나이: {age}</p>  
  </div>  
)
```



Hook – useEffect()

- `useEffect()`

디지털 시계

현재 시간: 오전 11:51:11



Hook – useEffect()

▪ useEffect()

```
export default function Clock(){  
  // time 상태 관리  
  const [time, setTime] = useState(new Date().toLocaleTimeString());  
  
  //처음 랜더링될때 타이머 설정  
  useEffect(() => {  
    const timer = setInterval(() => {  
      setTime(new Date().toLocaleTimeString());  
    }, 1000)  
    console.log("렌더링...");  
  }, [])  
  
  return(  
    <div>  
      <h2>디지털 시계</h2>  
      <h3>현재 시간: {time}</h3>  
    </div>  
  )  
}
```



Hook – useEffect()

- useEffect()

```
return (  
  <div>  
    <h2>디지털 시계</h2>  
    <h3>현재 시간: {time}</h3>  
  </div>  
);  
}
```



제어(Controlled) 컴포넌트

■ 회원 가입 폼

회원가입

이름

직업

성별 ☐ 남자 ☒ 여자

자기소개

안녕하세요~
이상한 변호사 우영우 입니다.

제출 데이터:

```
▼ {name: '우영우', job: '직장인', gender: '여자', text: '안녕하세요~  
영우 입니다.'} i  
gender: "여자"  
job: "직장인"  
name: "우영우"  
text: "안녕하세요~\n이상한 변호사 우영우 입니다."
```



리엑트 폼(Form)

■ 회원가입 폼 – SignUp.jsx

```
const SignUp = () => {  
  // 폼 데이터를 객체 형태로 관리  
  const [formData, setFormData] = useState({  
    name: '',  
    job: '직장인',  
    gender: '',  
    text: ''  
  })  
  
  // 모든 입력 필드의 변경을 처리하는 함수  
  const handleInputChange = (e) => {  
    const {name, value} = e.target;  
  
    setFormData({...formData, [name]: value})  
  }  
  
  // 폼 제출 시 데이터를 콘솔에 출력하는 함수  
  const handleSubmit = (e) => {  
    e.preventDefault(); // 폼 제출 시 페이지 새로고침 방지  
    console.log("제출 데이터: ", formData);  
  }  
}
```



리엑트 폼(Form)

■ 회원가입 폼 – SignUp.jsx

```
return(  
  <div className="sign-up">  
    <h2>회원가입</h2>  
    <form onSubmit={handleSubmit}>  
      <p>  
        <label htmlFor="name">이름</label>  
        <input  
          type="text"  
          name="name"  
          value={formData.name}  
          onChange={handleInputChange}  
          placeholder="이름을 입력하세요"  
        />  
      </p>  
      <p>  
        <label htmlFor="job">직업</label>  
        <select  
          name="job"  
          value={formData.job}  
          onChange={handleInputChange}  
        >  
          <option value="직장인">직장인</option>  
        </select>  
      </p>  
    </form>  
  </div>  
)
```



리엑트 폼(Form)

▪ 회원가입 폼 – SignUp.jsx

```
<option value="학생">학생</option>
<option value="프리랜서">프리랜서</option>
</select>
</p>
<p>
  <label htmlFor="gender">성별</label>
  <label><input
    type="radio"
    name="gender"
    value="남자"
    checked={formData.gender === "남자"}
    onChange={handleInputChange}
  />남자</label>
  <label><input
    type="radio"
    name="gender"
    value="여자"
    checked={formData.gender === "여자"}
    onChange={handleInputChange}
  />여자</label>
</p>
```



리엑트 폼(Form)

- 회원가입 폼 – SignUp.jsx

```
<p>
  <label htmlFor="text">자기 소개</label>
  <textarea
    name="text"
    value={formData.text}
    onChange={handleInputChange}
    rows={5}
    cols={10}
  />
</p>
<button type="submit">가입</button>
</form>
</div>
)
}

export default SignUp;
```



리엑트 폼(Form)

- App.css

```
.sign-up h2{  
  margin-bottom: 20px;  
}  
.sign-up form p{  
  margin-bottom: 10px;  
}  
.sign-up form label{  
  display: inline-block;  
  width: 80px;  
}  
.sign-up form input[type="text"],  
.sign-up form input[type="password"],  
.sign-up form select,  
.sign-up form textarea{  
  width: 200px;  
}
```



리엑트 폼(Form)

- 로그인 폼

로그인

user1

.....

로그인

환영합니다. user1님!

로그인

cloud

.....

로그인

아이디 또는 비밀번호가 일치하지 않습니다.



리엑트 폼(Form)

로그인 폼 – SignIn.jsx

```
// 임시 데이터(객체 리스트) 생성
const users = [
  {username: 'user1', password: 'user1111'},
  {username: 'user2', password: 'user2222'},
  {username: 'admin', password: 'admin0000'}
]

const SignIn = () => {
  // 객체 상태 관리
  const [formData, setFormData] = useState(
    {username: '', password: ''}
  )
  // 로그인 결과 상태 관리
  const [result, setResult] = useState(null);

  // 입력 필드 변경 핸들러
  const handleInputChange = (e) => {
    const {name, value} = e.target;
    // console.log(e);
    setFormData({
      ...formData,
      [name]: value //name 속성과 값
    })
  }
}
```



리엑트 폼(Form)

로그인 폼 – SignIn.jsx

```
const handleSubmit = (e) => {  
  e.preventDefault();  
  const {username, password} = formData;  
  //데이터 일치 여부 - find() 메서드  
  const matched = users.find((user) =>  
    user.username === username && user.password === password);  
  
  //로그인 성공여부에 따라 결과 상태 업데이트  
  setResult(matched ? 'success' : 'fail');  
}  
  
return(  
  <div className="sign-in">  
    <h2>로그인</h2>  
    <form onSubmit={handleSubmit}>  
      <p>  
        <input  
          type="text"  
          name="username"  
          value={formData.username}  
          onChange={handleInputChange}  
          placeholder="아이디를 입력하세요"  
        />  
      </p>  
    </form>  
  </div>  
)
```



리엑트 폼(Form)

로그인 폼 – SignIn.jsx

```
<p>
  <input
    type="password"
    name="password"
    value={formData.password}
    onChange={handleInputChange}
    placeholder="비밀번호를 입력하세요"
  />
</p>
<button type="submit">로그인</button>
</form>
{/* 결과 메시지 출력 */}
{result === 'success' && (
  <p className="result-msg" style={{color: 'blue'}}>
    환영합니다. {formData.username}님!
  </p>
)}
{result === 'fail' && (
  <p className="result-msg" style={{color: 'red'}}>
    아이디 또는 비밀번호가 일치하지 않습니다.
  </p>
)}
</div>
)
```



실습 문제

- 문제

useEffect를 사용하여 컴포넌트가 처음 실행될 때 콘솔 출력.

페이지 로딩...



실습 문제

- 문제

입력값을 관리하는 회원가입 폼을 만드세요.

이름
이메일
비밀번호



실습 문제

- 문제

사용자 리스트를 map으로 출력하세요

```
[  
  {id:1, name:"홍길동"},  
  {id:2, name:"이순신"}  
]
```



실습 문제

- 문제

useState로 좋아요 버튼을 만드세요.

좋아요 0 → 1 → 2



실습 문제

- 문제

입력창에 입력한 내용을 실시간으로 화면에 출력하세요.

